01. Collaborative Pi Durable



Target time: 15 seconds

Speaking points

- I extended Pi Durable itself so several people can instruct shared coding agents in the same project folder. The central engineering work is deciding what gets saved, how shared requests are ordered, and what recovery can safely repeat.
- This is a take-home prototype built with AI assistance and reused upstream code. I will distinguish implemented behavior, measured evidence, and planned extensions.

Suggested demonstration: No interaction yet. Keep two authenticated browser sessions ready on the future GCP host and use a disposable presentation thread/store.

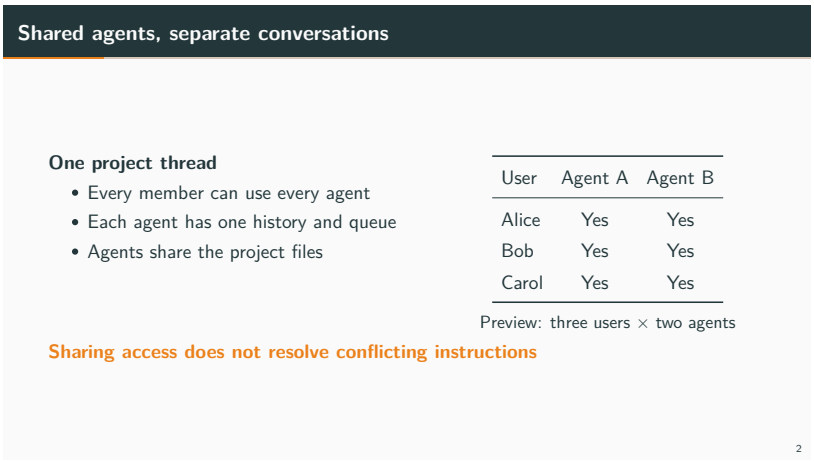
Fallback: Use the included local screenshot and controlled file trace if the model or hosted app is unavailable.

Boundary to state: GCP is not deployed. All cloud demonstrations in these notes are a post-deployment plan, not a record of a completed cloud run.

References: Wiki: the short project pitch

Source: PROJECT_INSTRUCTIONS.txt

02. Shared agents, separate conversations



Target time: 25 seconds

Speaking points

- M:N means users share agent identities, not that every person gets a private copy of each agent. Alice and Bob can both address A and B; A has one shared conversation and queue, and B has a separate conversation.
- Both agents see the same workspace files. The three-by-two table illustrates the relationship model and matches the deterministic preview; the normal real-provider example has two users and two agents.
- The system can record Alice asking for React and Bob asking for Vue. Ordering and attribution do not decide which product requirement should win.

Suggested demonstration: On the future GCP app, show the member list, both agent panels, and the access/coordination section. Use Alice and Bob for the live four pairings; add Carol only in a separately configured fresh store if you want six pairings.

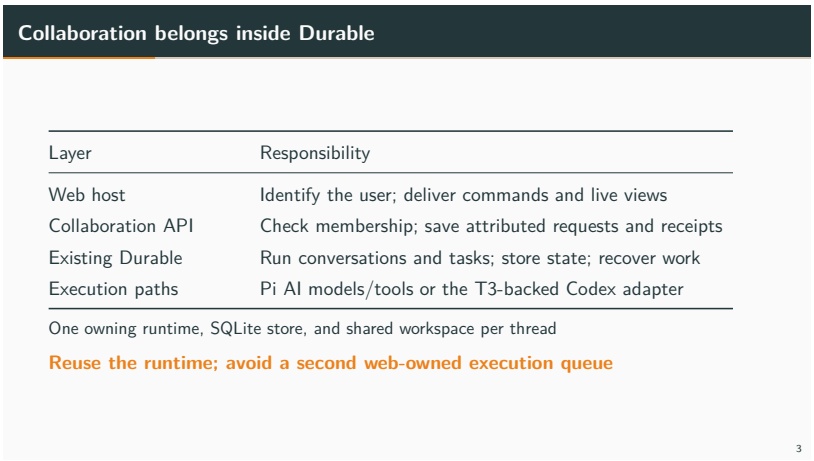
Fallback: Point to the access matrix and explain why two pages observe the same agent history.

Boundary to state: Membership is fixed for a saved store. The matrix shows access, not online presence or per-agent privacy; agents do not automatically share every conversation message.

References: Wiki: many to many sharing is a relationship model

Wiki: why do two agents share files but keep separate conversation histories

03. Collaboration belongs inside Durable



Target time: 60 seconds

Speaking points

- The host identifies Alice from her login token. The collaboration package checks membership and saves her attributed request; existing Durable conversations, inboxes, tasks, and storage run the work.
- The key design decision was avoiding a second queue and database in the web server. Otherwise the server could accept a request and crash before handing it to Durable. Saving the receipt and work together removes that particular gap.
- Pi AI and native coding tools were reused. The optional external path uses a T3-backed Codex adapter. Chord remains inside Durable, so this is a flexible package boundary, not a claim that all framework dependence was removed.

Suggested demonstration: Keep this architectural explanation on the slide. Optionally show that Alice's and Bob's browser pages use the same host; a standalone package example can exercise collaboration without the web server.

Fallback: Trace Alice -> authenticated host -> collaboration package -> Durable -> native tools or Codex -> shared workspace.

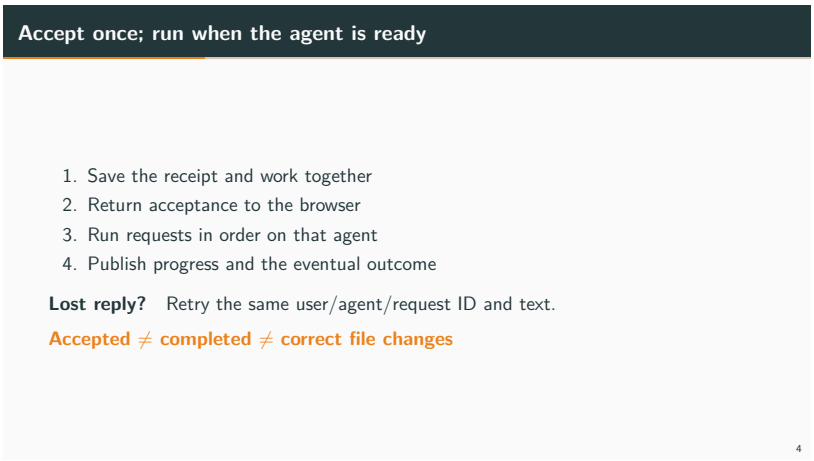
Boundary to state: The current model is one exclusive owner per thread store/workspace. The browser and read-only file viewer do not own the agent execution queue.

References: Wiki: why the feature lives inside durable

Wiki: what was added and what was reused

Wiki: thread conversation lane session and harness

04. Accept once; run when the agent is ready



Target time: 40 seconds

Speaking points

- Acceptance is a short saved-state decision; the model can take much longer. The HTTP 202 reply means the request was accepted, not that the requested code is correct. Work can begin after the save before that reply reaches the browser; the slide lists conceptual stages, not a promise to wait for network delivery.
- If the response is lost after saving, Alice retries the same thread/user/agent/request ID and text to find the original receipt. A new ID is new work; the same identity with changed text is rejected.
- The demo runs follow-ups one at a time on each agent. Independent agent lanes can overlap, while database commits take turns separately; neither rule protects all shared-file changes.

Suggested demonstration: If the first live instruction is still active, submit Bob's prepared follow-up to the same agent and point to queued status. Do not rely on typing speed. To show an actual ambiguous-response retry, use a separately rehearsed request harness preserving the exact original ID; there is no current GUI fault-injection button.

Fallback: Use the click-to-result sequence diagram in the linked wiki. A stable-ID retry test is more reliable than manually interrupting a fast live request.

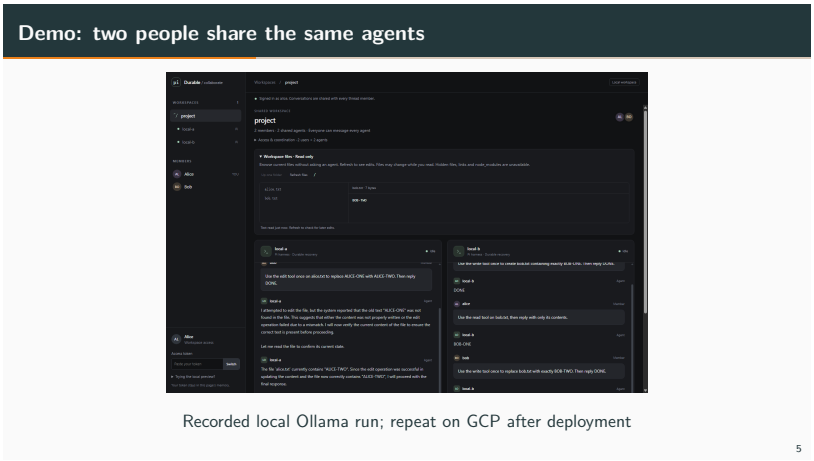
Boundary to state: Do not equate one accepted command with one model request or one file effect. Browser reload/relogin can lose its unsent or ambiguous local retry draft.

References: Wiki: a command from click to result

Wiki: retry identity and idempotent admission

Wiki: idempotency deduplication and exactly once claims

05. Demo: two people share the same agents



Target time: 105 seconds

Speaking points

- I will show shared observations and actual file bytes. A model's DONE message is not my test of success.
- The screenshot is from the recorded local Ollama run, not from GCP. It also shows a model describing an edit mismatch despite correct final bytes, which is why independent file inspection matters.
- Use the configured agent IDs shown in the app; A and B are explanatory names. Local Ollama can process inference serially even though agent histories and runtime lanes are independent.

Suggested demonstration: After GCP readiness: open two browser profiles and privately authenticate as Alice and Bob. Alice -> A: 'Use the write tool once to create alice.txt containing exactly ALICE-ONE. Then reply DONE.' Bob -> A: 'Use the edit tool once on alice.txt to replace ALICE-ONE with ALICE-TWO. Then reply DONE.' Bob -> B: 'Use the write tool once to create bob.txt containing exactly BOB-ONE. Then reply DONE.' Alice -> B: 'Use the read tool on bob.txt, then reply with only its contents.' Inspect ALICE-TWO and BOB-ONE in Workspace files, using manual Refresh. Reconnect/reload Alice and sign in again if needed; show saved history without resubmitting. Paste prompts prepared in advance.

Fallback: Use the screenshot and the recorded local browser evidence. If A finishes before Bob submits, explain that no wait was needed; do not claim that run demonstrated queueing. Use controlled tests to establish the queue rule.

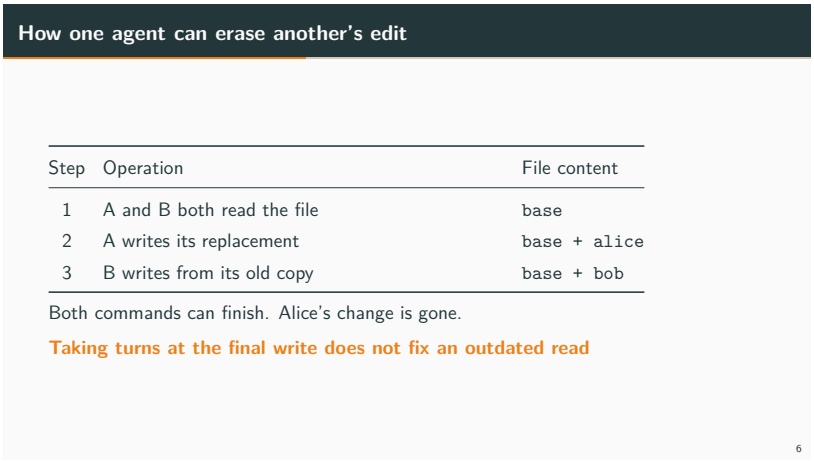
Boundary to state: Use a fresh disposable presentation store and rehearse these filenames. Do not overwrite a live team's files or claim that a screenshot establishes cloud deployment. Refresh is manual and the file view is not an atomic snapshot.

References: Wiki: what the current gui can prove

Wiki: the file browser is inspection not version control

Source: apps / collab demo / scripts / results / 2026 11 08 ollama browser.json

06. How one agent can erase another's edit



Target time: 35 seconds

Speaking points

- Both agents can read the same old base before either writes. A saves base plus Alice, and B later saves base plus Bob from its stale copy, erasing Alice's change.
- The final writes happened in order, so serializing individual writes is insufficient. The gap is read, model planning, then write; shell commands and external Codex can also bypass Pi's individual mutation gate.
- This is a controlled lost-update example. Unrestricted real writes may also fail or leave partial content, so last-writer-wins is not a universal filesystem promise.

Suggested demonstration: There is no current GUI control to pause both agents after reading. On GCP, use the rehearsed controlled test/example from an SSH terminal or show this trace; do not race two real prompts and assume their ordering is known.

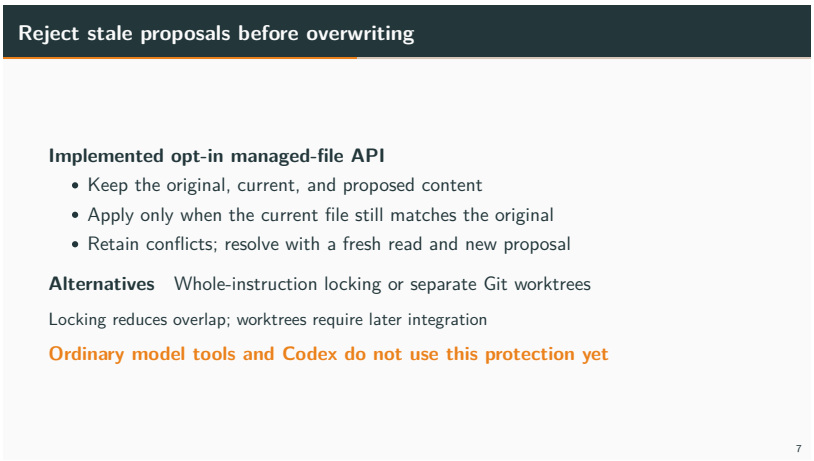
Fallback: The table is the entire causal trace. Ask the audience what B's remembered file contains after A writes.

Boundary to state: Ordinary model tools still permit this risk. Same-agent queues are not cross-agent conflict resolution.

References: Wiki: the lost update example to remember

Wiki: javascript runs on one event loop so how can two agents still lose a file edit

07. Reject stale proposals before overwriting



Target time: 40 seconds

Speaking points

- The managed-file API borrows the database idea of a conditional update: apply only if the file still matches what this participant originally read. It preserves base, current, and proposed content when the check fails.
- Alice applies first, Bob's stale proposal conflicts, then a fresh proposal explicitly combines both changes. No automatic merge is claimed, and repeated old proposal IDs retain their recorded outcomes.
- This is an opt-in single-file API, not the normal model-tool path. Whole-instruction ownership is an alternative with less concurrency; separate Git worktrees isolate edits but require later integration.

Suggested demonstration: After building on the VM, run example 41 in a separate SSH terminal: cd /opt/pi/packages/durable; node --conditions=source --experimental-strip-types test/examples/41-managed-file-conflict.ts. It owns temporary files, needs no app server or model key, and shows applied/conflict/explicit-resolution results. A future GUI panel could expose the same steps, but no such panel exists today.

Fallback: Use the managed-file diagram in the wiki or a previously recorded example result, clearly labelled as recorded evidence.

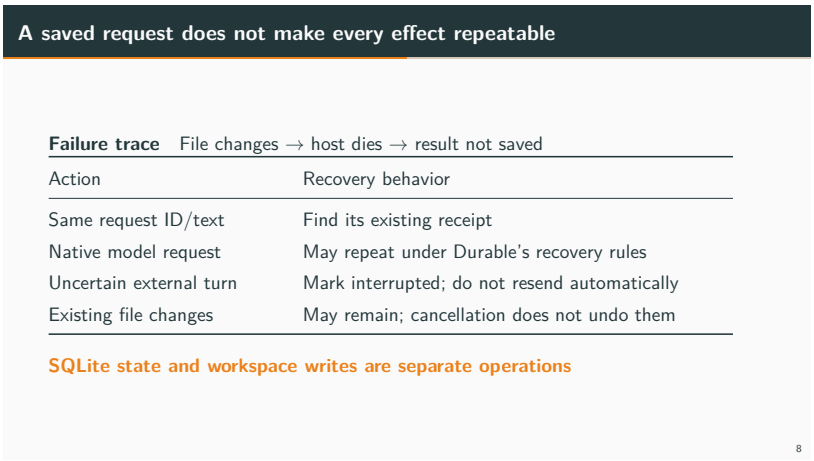
Boundary to state: All relevant writers must participate. The current slice has no automatic merge, multi-file transaction, or automatic uncertain-write reconciliation; ordinary native tools, shell, Codex, and the GUI are not integrated with it.

References: Wiki: managed publication and optimistic concurrency control

Wiki: what git contributes and what it does not

Source: packages / durable / test / examples / 41 managed file conflict.ts

08. A saved request does not make every effect repeatable



Target time: 60 seconds

Speaking points

- Suppose a tool appends hello and the host dies before saving success. A request receipt tells us the instruction was accepted, not whether the append happened; blindly repeating it could append twice.
- Native generation retains Durable's recovery rules, including possible repeated model requests and declared unsafe-tool no-replay. The external path saves a dispatch marker before attempting Codex and reports an uncertain attempt interrupted instead of automatically resending.
- The trade-off is that a crash after the marker but before send can interrupt work that never started. Cancellation does not undo existing file changes, and a stopped parent process does not prove every child writer has stopped.

Suggested demonstration: A safe hosted GUI demonstration is reconnect followed by a rehearsed idle service stop/start on a disposable store, then restored files/history. Keep active crash injection and external-effect tests separate in controlled fixtures; never kill the shared live host for an unrehearsed presentation.

Fallback: Walk through the append/crash diagram in the glossary. State separately what happens to the receipt, model request, and file effect.

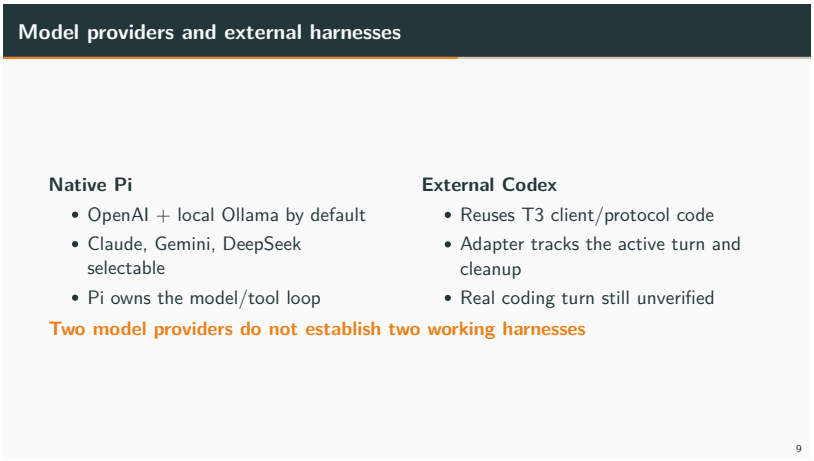
Boundary to state: An idle restart is not active-shell takeover or power-loss proof. Storage errors can happen after commit; the whole failed Harness must be stopped before reopen. Windows Bash shutdown investigation remains deferred.

References: Wiki: llm wiki / 03 durability and recovery.md

Wiki: side effects checkpoints and dispatch markers

Wiki: llm wiki / interview questions / 04 failures and recovery.md

09. Model providers and external harnesses



Target time: 35 seconds

Speaking points

- A provider supplies model output; a harness owns the surrounding conversation and tool loop. OpenAI plus Ollama uses two providers through Pi, whereas Codex is an external harness with its own loop.
- The Codex adapter actually executes vendored T3 typed client/protocol code, with replay support in tests and preserved provenance/licenses. It does not import T3's whole application, credential system, or UI.
- Scripted subprocess tests caught wrong-turn correlation and interrupt-delivery/cleanup races. Real CLI initialization works, but no recorded authenticated real Codex coding turn or continuation exists. Claude, Gemini, and DeepSeek have selectable provider paths; their live keys were not supplied for multi-key testing.

Suggested demonstration: On the future GCP host, show the configured OpenAI/Ollama agent names and use their existing successful instructions. Demonstrate a Codex lane only after separate service-user authentication and a real turn/continuation check; use a fresh store for changed agent configuration.

Fallback: Use the provider/harness diagram and the T3 provenance file instead of pretending a native OpenAI turn is Codex evidence.

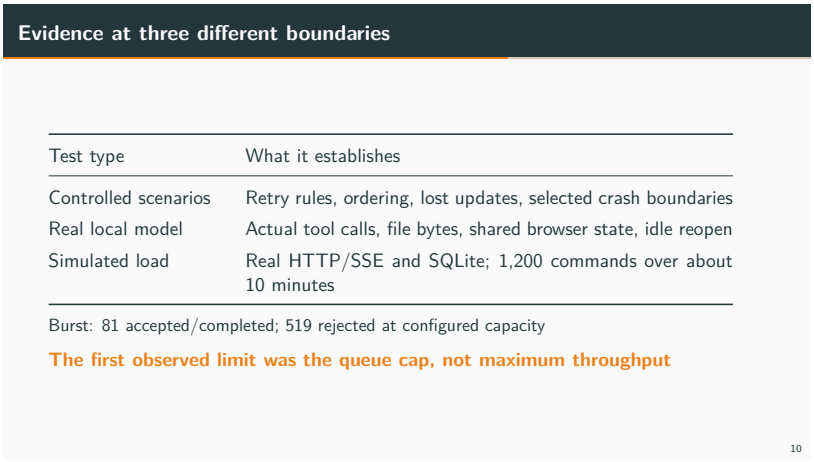
Boundary to state: Unsupported interactive Codex approvals/questions fail visibly; shared approval arbitration is not implemented. OpenAI API credentials and Codex authentication are separate prerequisites.

References: Wiki: exactly what t3 contributed

Wiki: provider model agent harness and adapter

Source: packages / durable / src / adapters / codex / vendor / UPSTREAM.md

10. Evidence at three different boundaries



Target time: 40 seconds

Speaking points

- Controlled tests force races and selected crash points; real local-model checks establish actual tool use and file bytes; load checks exercise real transport/storage with simulated work. These are different claims, and no category replaces the others.
- The roughly ten-minute soak completed 1,200 commands at an offered two per second across ten lanes. The burst accepted/completed 81 and rejected 519 at configured capacity, so the first limit observed was the queue cap, not a database throughput ceiling.
- Across baseline, burst, and soak, all 1,301 accepted logical commands had one recorded start; 30 retry probes added no starts. These are observations for simulated execution, not a universal exactly-once or real-model throughput guarantee.

Suggested demonstration: Show the portable load report from the repository or a saved report in the SSH terminal. The current web app has no performance dashboard. Running the workload in another disposable owner can be rehearsed separately; avoid load against the presentation's live data.

Fallback: Use the slide's measured counts and explicitly describe simulated execution over real SQLite/HTTP/SSE.

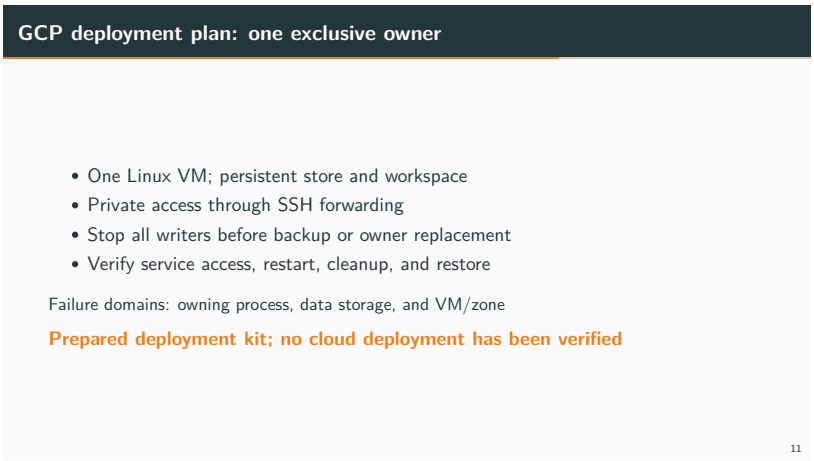
Boundary to state: The recorded p95 was 236.6 ms and sampled peak RSS 179.2 MiB for a combined client/server process; neither is a bound. The latest full recorded demo suite covered 38 cases, with later focused runs; do not claim a fresh all-41 run. OpenAI currently works by the user's manual report.

References: Wiki: the load result worth remembering

Wiki: what the measurements cannot prove

Source: apps / collab demo / load / README.md

11. GCP deployment plan: one exclusive owner



Target time: 30 seconds

Speaking points

- A single Linux VM fits the prototype's long-lived process, SQLite, and shared local workspace. Data is on persistent storage; the app binds to loopback and evaluator browsers connect through SSH forwarding.
- One owner is an explicit limitation: process, data storage, and VM/zone are failure domains. More web-server copies against the same workspace do not safely provide high availability.
- The prepared unit disables automatic restart for inspection and requests group cleanup. Actual Linux service authentication, descendant shutdown, missing-mount behavior, reboot persistence, and backup restoration still need verification.

Suggested demonstration: After deployment: gcloud compute ssh VM_NAME --project=PROJECT_ID --zone=ZONE -- -N -L 4310:127.0.0.1:4310. Then open http://127.0.0.1:4310 in two profiles. The browser URL is local because the SSH tunnel forwards to GCP. Keep tokens and provider keys off the projected screen.

Fallback: Explain the prepared deployment runbook and mark every cloud acceptance check pending. There is no GCP URL, VM, or recorded cloud screenshot to show yet.

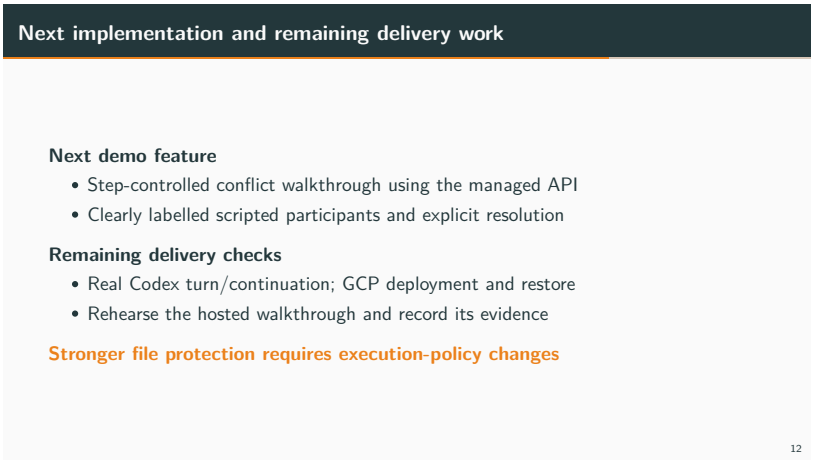
Boundary to state: Do not expose the app publicly as a shortcut. The current local Ollama result does not establish model performance on a small CPU VM; choose and test the VM/model composition before the talk.

References: Wiki: why the proposed gcp deployment uses one vm

Wiki: llm wiki / interview questions / 08 scaling and operations.md

Source: apps / collab demo / deploy / README.md

12. Next implementation and remaining delivery work



Target time: 25 seconds

Speaking points

- The next useful presentation feature is a bounded, step-controlled managed-file conflict panel. It would show both original reads, Alice's accepted proposal, Bob's rejected stale proposal, and a fresh explicit resolution.
- That panel uses scripted participants and requires no additional model key or GCP. Binding the protection to ordinary agent tools, restricting bypass writes, and handling uncertain publication are separate execution-policy work.
- Real Codex evidence, cloud deployment/restore, and human rehearsal remain delivery checks. Longer-term options include roles/approvals, worktrees, fair budgets, and exclusive workers for independent threads; they are outside this immediate slice.

Suggested demonstration: Return to the future deployed app for questions, but do not point to a conflict panel that does not exist. If it is implemented later, re-record its evidence and revise these notes before claiming it works.

Fallback: End on the concrete next slice and its boundaries, then use backup slides only for relevant questions.

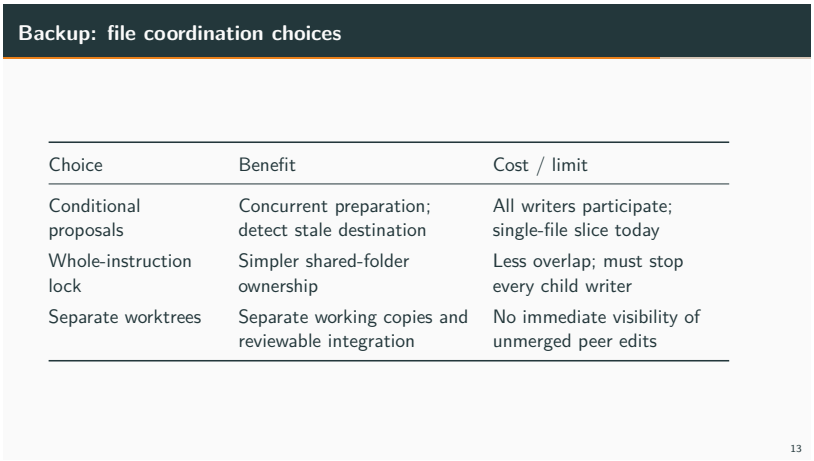
Boundary to state: The whole assignment is not complete while cloud delivery and the stated verification gates remain pending.

References: Wiki: how the proposed scenario panel improves the story

Wiki: what would you implement next and what would you defer

Source: current risks and open checks

13. Backup: file coordination choices



Target time: Backup slide

Speaking points

- Optimistic checks allow independent preparation and reject an outdated destination at publication. This protects cooperative writes but does not validate every file used by a plan, and exact text checks do not detect content changing away and back.
- Whole-instruction ownership is simpler for unrestricted harnesses but requires every writer to honor the rule until all child processes stop. A lease expiring is not enough to stop an old shell.
- Separate worktrees create separate working files; branches alone do not. Integration needs review, merge, and testing, and changes the assignment's immediate shared-folder visibility. T3's inspected startup workspace lease ends before the provider turn; it is not a whole-turn lock.

Suggested demonstration: Use example 41 in a separate VM terminal for the implemented conditional-write choice. Discuss worktrees and whole-instruction locking as designs, not hidden modes of the current app.

Fallback: Work through base/current/proposed and ask which trade-off best fits a requirement for unrestricted shell tools.

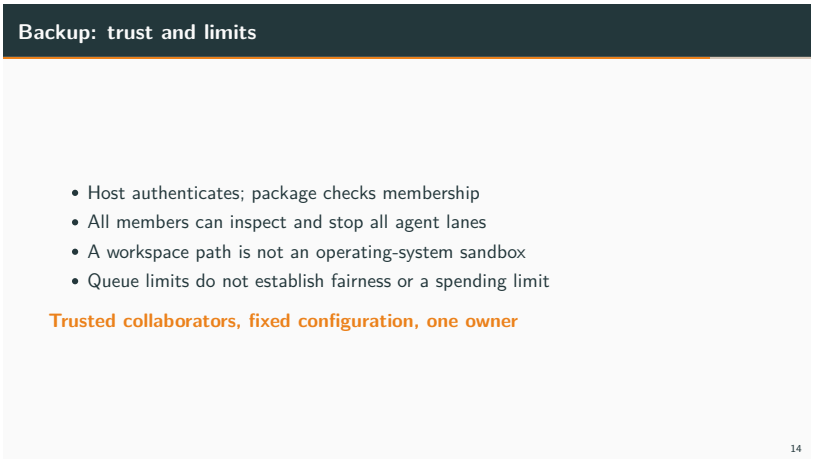
Boundary to state: No automatic Git stash/pop, merge engine, global workspace lock, or multi-file transaction is implemented.

References: Wiki: llm wiki / interview questions / 03 file conflicts.md

Wiki: locks ownership leases and fencing

Source: apps / collab demo / CONFLICT_RESOLUTION_PROPOSAL.md

14. Backup: trust and limits



Target time: Backup slide

Speaking points

- Authentication maps the bearer token to Alice; membership authorization is enforced inside the package. Passing the string alice to the library is not authentication, and a prompt claiming to be an administrator carries no additional authority.
- All thread members intentionally see all lanes and can stop peer work. Fixed membership has no dynamic revocation flow, and the trusted coding process can access credentials available to its service account.
- The default limits include 256 lifetime commands and 16 pending per agent, but one user can consume capacity and one command can call a model many times. A public service needs a different identity, isolation, budget, and retention design.

Suggested demonstration: If asked, show the GUI's shared scope with Alice and Bob. Use existing automated unauthorized-access tests as evidence instead of displaying real tokens or attempting destructive commands.

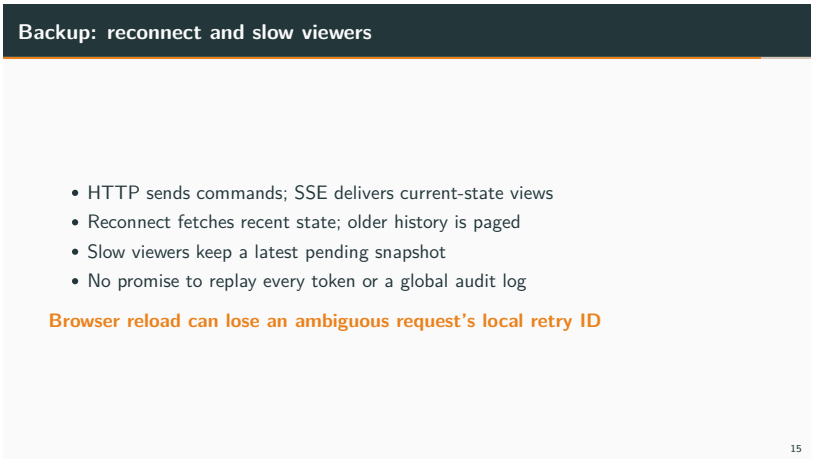
Fallback: Contrast host identity checks with package membership, then explain why neither is an OS sandbox.

Boundary to state: Path/link checks in the read-only viewer are hygiene for trusted users, not confinement against hostile concurrent writers or prompt injection.

References: Wiki: authentication membership and authority

Wiki: llm wiki / interview questions / 06 security and api.md

15. Backup: reconnect and slow viewers



Target time: Backup slide

Speaking points

- The browser gets current-state snapshots over SSE, not a durable stream of every intermediate token. It can page older committed messages, and stable entry IDs avoid duplicate display after reconnect.
- One shared watch observes each lane; a blocked viewer keeps only a latest pending snapshot. Limits constrain viewers, frames, and application buffering, not all kernel buffers or total process memory.
- A browser can forget an ambiguous draft's retry ID on reload even while the server retains accepted work. A future saved client receipt/draft list would need explicit retention and reconciliation, without storing bearer tokens.

Suggested demonstration: Disconnect only Alice's browser connection, let Bob observe accepted work, reconnect Alice and inspect history. Reauthenticate after reload if needed; do not resend the instruction under a new ID just to restore the display.

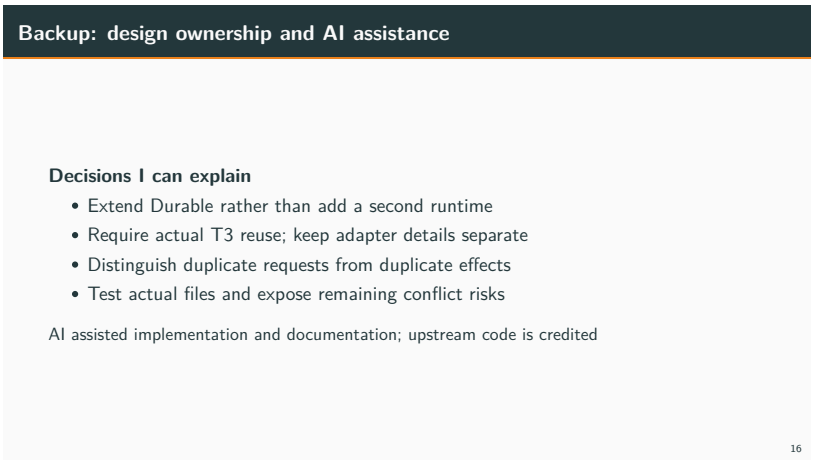
Fallback: Explain that a latest-view stream can omit progress steps while still catching up with completed messages and state.

Boundary to state: The views combine sources and are not an atomic cross-source snapshot or a global audit log. No GUI network-fault toggle or durable replay cursor is provided.

References: Wiki: snapshots are convergent views not an event journal

Wiki: backpressure fan out and coalescing

16. Backup: design ownership and AI assistance



Target time: Backup slide

Speaking points

- I directed the scope toward evolving Durable itself, required real T3 reuse, challenged whether the demo truly showed M:N sharing, and kept the file-conflict limitations visible. AI assistance helped implement, investigate, and document the work.
- I should be able to explain a request, a crash boundary, and a lost-update trace without relying on an assistant's summary. Existing Pi storage/tools/providers and T3 client code remain credited upstream work.
- The original approximately twenty-hour target shaped prioritization, but it is not a measured claim about actual elapsed development time. I would not claim I manually wrote all code or that passing tests substitute for understanding it.

Suggested demonstration: No new app interaction is needed. Answer a concrete follow-up using a source file, wiki explanation, or the controlled example rather than opening a large unprepared code tour.

Fallback: Explain one trade-off you deliberately accepted and what evidence would justify changing it.

Boundary to state: Treat these as preparation points to phrase truthfully in your own words; do not invent a personal development history or an unrecorded result.

References: Wiki: how do you explain ai assisted development

Wiki: how would you explain your own judgment when much of the implementation used ai assistance